Cell Therapy Ticket Triage

AI-powered IT ticket triage for cell therapy manufacturing scheduling systems

Live App: https://finalproject-connormcguire.streamlit.app/

Context, User, and Problem

User: A Senior Business Systems Manager at a cell therapy manufacturing company who receives IT support tickets from master schedulers across multiple global sites. These schedulers manage access to manufacturing slots for made-to-order cell therapies — a time-critical operation where system issues can directly impact patient treatment timelines.

Workflow: The manager manually reads each incoming ticket, decides whether it warrants immediate escalation to senior IT resources or can be delegated to standard support, and if escalating, drafts two notification emails — one to the business user, one to the IT resource. This happens across a high volume of tickets with inconsistent urgency signals.

Why it matters: A missed escalation in this environment is not just an inconvenience. Scheduling system failures that go un-escalated can delay patient treatment. The manager needs a reliable, fast first-pass triage that flags the right tickets every time — not a tool that makes the final call, but one that surfaces what needs attention and drafts the communications to act on it.

Solution and Design

Analyzes incoming IT support tickets and classifies them as **ESCALATE** or **DELEGATE** using a six-question rubric. For escalated tickets, the app automatically drafts two emails — one to the business user and one to the IT resource.

Triage Rubric

A ticket is escalated if any of the following are true:

1. More than one user or site is affected
2. The ticket describes a system bug or data error affecting multiple users
3. The ticket contains patient, clinical, or treatment language in a disruption context
4. The ticket requests changes to slot capacity, approval workflows, or site configuration
5. The issue has occurred before or is recurring
6. The ticket was submitted by a VIP user

How the AI Evaluates Tickets

Each rubric question is sent to the AI model as a **separate, independent API call**. The model sees the ticket and one question at a time — it has no visibility into how it answered the other five questions when making each individual judgement.

This design is intentional. A single prompt asking the model to evaluate all six criteria at once risks anchoring bias, where an early strong signal colors how the model interprets later questions. By isolating each question into its own call, every criterion is evaluated on its own merits, producing six independent YES/NO verdicts before a final classification is made.

If any single question returns YES, the ticket is classified as ESCALATE — mirroring how a human reviewer should work through the rubric.

Key Design Choices

Model selection: Claude Haiku was chosen over larger models after explicitly weighing cost and latency against accuracy needs. Each ticket generates 7 API calls (6 rubric + 1 email draft). Haiku handles binary YES/NO classification accurately at a fraction of the cost of Sonnet or Opus, making batch processing practical.

Structured output constraints: Each rubric call uses max_tokens=10 and instructs the model to respond only YES or NO — eliminating verbose reasoning and making outputs reliable enough to parse programmatically. Email drafting uses a strict format template with required subject line prefixes and a mandatory closing sentence.

Human stays in the loop: The app never sends emails. All ESCALATE classifications are presented as recommendations. The manager makes the final call — the tool accelerates the workflow, it does not replace the judgement.

Course Concepts Integrated

Model and provider selection with cost/latency/quality trade-offs

Claude Haiku was chosen over larger models after explicitly weighing cost and latency against accuracy needs. Each ticket generates 7 API calls (6 rubric + 1 email draft). Haiku delivers sufficient accuracy for binary YES/NO classification at a fraction of the cost of Sonnet or Opus, making batch processing practical without sacrificing triage quality.

Anatomy of an LLM call: system prompts, output constraints, structured outputs

Every rubric call uses max_tokens=10 and instructs the model to respond only YES or NO — eliminating hallucinated reasoning and making outputs reliable enough to parse programmatically. Email drafting uses a strict format template with required subject line prefixes and a mandatory closing sentence. A shared system prompt establishes the business context for every call.

Context engineering

The system prompt provides domain-specific context the model could not infer on its own — what master schedulers do, why slot access is time-critical, and what the organizational stakes are. This context is passed on every call so the model evaluates tickets against the right frame of reference rather than a generic IT support context.

Evaluation design: rubrics, test sets, baselines

A 15-ticket synthetic test set was built specifically to evaluate the tool against a target escalation rate. Tickets were designed to hit specific rubric questions and cleanly avoid others. Results were compared against the manual baseline on accuracy, consistency, and time. Over-escalation on Q2 and Q4 was identified through testing — a real evaluation finding, not a cherry-picked demo.

Governance and deployment controls: human review, action limits, logging

The app never sends emails automatically. All classifications are framed as recommendations. The manager reviews every draft before acting. Escalation and delegation logs are maintained and exportable for audit. The system is explicitly designed so that a human makes every consequential decision.

Evaluation and Results

Baseline Comparison

The baseline is the current manual process: the manager reads each ticket individually, applies the rubric from memory, decides whether to escalate, and if so drafts two emails from scratch. Dozens of tickets are submitted over the course of a week.

Dimension	Manual Baseline	This Tool
Time for 15 tickets	~45 minutes	30–60 seconds
Triage consistency	Variable — memory-based, fatigue affects judgement	Consistent — same six questions every time
Email drafting	Written from scratch per escalation (~5 min each)	Auto-drafted instantly, ready for review
Rubric coverage	Risk of skipping criteria under pressure	All six criteria evaluated every time

At roughly 5 minutes per escalated ticket (read, decide, draft two emails) and 2 minutes per delegated ticket, manually triaging a batch of 15 tickets takes approximately 45 minutes — assuming 5 escalations and 10 delegations. The app completes the same batch in 30–60 seconds. Across dozens of tickets per week, that compounds into hours of recovered time. The consistency gain matters equally — the rubric is applied the same way on every ticket, regardless of workload or fatigue.

Test Set and Results

A set of 15 realistic synthetic tickets was designed to produce approximately 25% escalation rate. Tickets were crafted to hit specific rubric questions — multi-site outages, patient impact language, recurring issues, VIP users — as well as clean DELEGATE cases.

The model consistently identified the designed escalations. Over-escalation occurred on tickets with approval workflow language (Q4) and system behavior descriptions (Q2). After iterating on ticket language, the tool reached the target escalation rate, revealing that Q2 and Q4 are the most sensitive criteria and most prone to false positives.

Where It Works

- Clear multi-site or multi-user outages (Q1)
- Explicit patient/clinical/treatment language (Q3)
- Stated recurring history (Q5)
- VIP user identification (Q6)

Where It Fails or Needs Human Review

- Tickets with vague system behavior language that could be one-user or multi-user
- Tickets that mention workflows or approvals in passing without requesting a config change
- Nuanced cases where escalation depends on organizational context the model does not have
- Email drafts always require review before sending — tone and specifics may need adjustment

---

Setup and Usage

Running on Streamlit Cloud

The app is live at https://finalproject-connormcguire.streamlit.app/ — no installation required. To provide an API key for grading, add it under App Settings → Secrets in Streamlit Cloud:

```
ANTHROPIC_API_KEY = "your-key-here"
```

Running Locally

1. Clone the repo
2. Install dependencies: pip install -r requirements.txt
3. Set your Anthropic API key: $env:ANTHROPIC_API_KEY='your-key-here' (Windows PowerShell)
4. Run the app: python -m streamlit run streamlit_app.py
5. Upload sample_tickets_7.csv using the Batch CSV Upload tab to see a full triage run

---

Tech Stack

- Streamlit — UI framework
- Anthropic Claude (claude-haiku-4-5) — AI classification and email drafting
- openpyxl — Excel export